Client Projects
Services Guide

Ujjwal Prakash • Senior Software Engineer & Team Lead • IIT Kanpur
ujjwalprakash858@gmail.com • LinkedIn • GitHub

How I build software for clients. A transparent breakdown of my technical stack, architecture principles, engineering process, and what working with me actually looks like — so you can make an informed decision before we start.

Who I Am

I am a Senior Software Engineer and Team Lead at IIT Kanpur, currently leading the development of the official IIT Kanpur Recruitment Automation System (RAS) — a production platform serving over **30,000+ students and 20,000+ active users**. I manage a team of 4 engineers across a 37,000+ line production codebase.

Beyond engineering, I am a peer-reviewed researcher. My deep learning work on ISRO's earth observation data was **accepted for an oral presentation at IEEE IGARSS 2026** — one of the most competitive geospatial intelligence conferences in the world.

I also teach full-stack engineering, with a **4.8/5.0 student rating**, which means I can communicate complex technical decisions in plain language — something clients consistently value.

What I Build

Backend Systems

I specialise in high-concurrency, production-grade backend systems. This includes REST and gRPC API design, authentication flows, database schema design, background job processing, and systems that need to handle thousands of concurrent users without degrading.

- **Go (Golang)** — primary backend language for performance-critical services
- **TypeScript / Node.js** — APIs, microservices, BFF (Backend for Frontend) layers
- **PostgreSQL** — relational data modelling, query optimisation, indexing strategies
- **Redis** — caching, session management, pub-sub, distributed locks
- **Docker & CI/CD** — containerised deployments, GitHub Actions pipelines

Full-Stack Platforms

When the project requires a complete product, I can deliver end-to-end:

- **Next.js** — SSR/SSG, API routes, auth integration, performance-optimised frontend

- **React** — component-driven UI with TypeScript, state management, animation
- **Database-to-UI pipelines** — from schema design through to rendered data tables

AI & Research Integration
With hands-on ML research experience (deep learning for satellite imagery, ISRO collaboration), I can integrate modern AI capabilities into products:

- LLM API integration (OpenAI, Anthropic, Google)
- Computer vision pipelines for custom datasets
- Data processing and analytics backends

Technical Stack

Layer	Technologies
Languages	Go, TypeScript, Python, SQL
Backend	Go (net/http, Chi, Gin), Node.js, REST, gRPC
Frontend	Next.js 14/15, React, Tailwind CSS, Framer Motion
Databases	PostgreSQL, Redis, SQLite
DevOps	Docker, GitHub Actions, Nginx, Linux servers
AI/ML	PyTorch, OpenCV, Scikit-learn, LLM APIs
Tools	Git, Postman, pgAdmin, Figma (handoff)

How I Work — Engineering Process

Phase 1: Discovery (Days 1–3)
Before writing a single line of code, I need to fully understand what you are building and why. This phase involves a structured conversation covering your users, scale expectations, existing systems, and non-negotiables (compliance, latency, budget).

Deliverable: a written scope document with agreed-upon architecture decisions, tech stack rationale, and a realistic timeline.

Phase 2: Architecture & Schema Design (Days 3–7)
Good systems are designed before they are built. I produce:

- Entity-relationship diagrams and database schema
- API contract (endpoints, request/response shapes, error codes)
- Sequence diagrams for critical user flows
- Deployment architecture diagram (services, external dependencies, data flow)

This phase is deliberately slow. Decisions made here are far cheaper to change than decisions made in production.

Phase 3: Development (Variable)
I follow a trunk-based development workflow with atomic commits and descriptive pull requests. Every feature ships with:

- Unit and integration tests
- Clear inline documentation
- Migration scripts for any database changes
- A staging environment deployment before production

I send progress updates at every meaningful milestone — not just at the end.

Phase 4: Testing & Handoff
Before marking work complete, the system passes:

- Load testing at 2× expected peak traffic
- Security review (auth, input validation, SQL injection checks)
- Documentation: README, API reference, deployment runbook

Handoff includes a walkthrough call and 14 days of post-launch support for critical bugs at no additional charge.

What You Can Expect From Me

- **Honest timelines.** I do not promise what I cannot deliver. If a scope is unrealistic, I say so before we begin.
- **Clear communication.** Written updates, not just verbal ones. Decision logs so you always know why something was built the way it was.
- **Production-first thinking.** Code that will survive real users, not just a demo environment. I have maintained a 99.9% uptime system in production — I build with that standard.
- **No lock-in.** You own everything. Clean, documented code that any competent engineer can continue after me.

Engagement Types

Type	Duration	Best For
Project-based	2–16 weeks	Defined scope with clear deliverables
Advisory retainer	Monthly	Ongoing architecture review, code review, technical direction
Rescue mission	1–4 weeks	Existing codebase in trouble — performance, bugs, or security
Build to hand off	4–12 weeks	Full system built and documented for an in-house team to own

When We Are Probably Not a Good Fit

I want to be honest about this, because misaligned expectations waste everyone's time.

- You need a resource who is available 24/7 and interruptible on demand
- The project has no defined problem — only a vague idea with no user validation
- You expect a fixed-price quote before any discovery conversation
- The timeline is "as fast as possible" with no flexibility

How to Start

The first step is a 30-minute conversation — no pitch, no pressure. I want to understand your problem and tell you honestly whether I can help.

ujjwalprakash858@gmail.com

Ujjwal Prakash — Senior Software Engineer & Team Lead, IIT Kanpur. Promoted for leading a 30K+ user production system. IEEE IGARSS 2026 accepted researcher. 4.8/5.0 teaching rating.

linkedin.com/in/ujjwal-prakash-036873336 • github.com/ujjwalPrakash-spike